GUÍA PARA DESARROLLAR EN PLANTUML DIAGRAMAS DE CASOS DE USO DE ANÁLISIS

Caso base: Sistema de Agendamiento de Citas Medicas

Elemento	Descripcion
Asignatura	Análisis y Diseño de Software
Tema	Modelado de casos de uso de nivel 0 y apoyo al análisis orientado a objetos con PlantUML
Caso base	Sistema de Agendamiento de Citas mediante un Chatbot - RenaBot en el Centro Medico Renacer
Casos de Usos Seleccionados	Agendar cita en linea, Registrar paciente presencial, Gestionar disponibilidad medica y Consultar citas Agendadas
Requisitos Funcionales	Cuatro requisitos funcionales de nivel 0: RF-01, RF-02, RF-03 y RF-04 detallados en el SRS
Proposito	Orientar al estudiante para pasar desde requisitos funcionales de nivel 0 hacia casos de uso claros, trazables y verificables.
Grupo / NRC	Grupo 4 - Integrantes: Jonathan Ampuero, Victor Guano, Mario Rosas NRC: 36839
Fecha	19/10/2026

1. Propósito de la guía

Esta guía orienta el desarrollo de diagramas de casos de uso de análisis en PlantUML para el proyecto AgendaMed Renacer - Renabot, tomando como referencia una secuencia académica progresiva: partir del SRS de la propuesta 2(Bot de mensajería v3.1), seleccionar requisitos funcionales de nivel 0, identificar actores, modelar casos de uso principales y mantener trazabilidad entre requisito, actor, caso de uso y criterio de aceptación. El caso se limita a cuatro casos de uso de nivel 0 definidos por el grupo: Agendar cita en linea, Registrar paciente presencial, Gestionar Disponibilidad medica y Consultar citas agendadas, de manera que el resultado obtenido sea cada caso de uso tenga relación con las clases de analisis y diagramas de secuencia.

2. Relación con la ruta de análisis de la Unidad 1

Momento	Pregunta guía	Producto esperado	Relación con la siguiente tarea
U1T1 - Análisis funcional inicial	¿Qué funcionalidades de nivel 0 requiere el sistema y qué actores participan?	Diagrama de casos de uso en PlantUML y matriz RF-CU.	Permite identificar clases, atributos y responsabilidades para el análisis estructural.
U1T2 - Modelado estructural	¿Qué clases principales se derivan de los casos de uso?	Diagrama de clases de análisis con clases de entidad, control y frontera cuando corresponda.	Entrega objetos base para diagramas de secuencia.
U1T3 - Modelado dinámico	¿Cómo interactúan actores, objetos y clases para cumplir cada caso de uso?	Diagramas de secuencia derivados de los casos de uso y clases de análisis.	Cierra la trazabilidad requisito-caso-clase-secuencia.

3. Conceptos mínimos para modelar casos de uso

- **Actor:** Rol externo que interactúa con el sistema. En AgendaMed los actores son: Paciente, Recepcionista, Médico y Administrador.
- **Caso de uso:** Funcionalidad observable desde el punto de vista del actor. Debe nombrarse con verbo + objeto, por ejemplo: Agenda cita en linea.
- **Límite del sistema:** Caja o frontera que delimita qué funcionalidades pertenecen al sistema(AgendaMed) y qué elementos están fuera(plataforma de mensajeria).
- **Relación actor-caso de uso:** Comunicación directa entre un actor y una funcionalidad del sistema.
- **Relación <<include>>:** Se usa cuando un caso de uso siempre reutiliza un comportamiento obligatorio de otro.
- **Relación <<extend>>:** Se usa cuando un comportamiento opcional o condicionado amplía otro caso de uso.

4. Caso base: AgendaMed Renacer

El sistema debe permitir gestionar el ciclo completo de citas médicas de forma a traves de un bot conversacional 24/7 . Para ello, las 4 funcionalidades de nivel 0 operan sobre la siguiente estructura de datos mínima extraída del SRS.

Dato	Tipo sugerido	Descripción	Validación mínima
Paciente	Registro de persona	Persona que solicita la atencion medica mediante el bot o de manera presencial	Identificacion mediante cedula(10 digitos), pasaporte(8 a 9 alfanumericos) o RUC, nombres, telefono, correo
Cita	Registro transaccional	Reserva del paciente, medico, especialidad, sucursal, fecha y hora	Estado de la cita: Pendiente_pago, confirmada, pagada, asistida, cancelada
Disponibilidad	Agenda / franja horaria	Horarios disponibles en los que el medico atiende en una sucursal	Duracion por especialidad dentro del horario del centro medico en donde no hay cruces de horarios
Medico	Registro de personal	Profesional que atiende al paciente dependiendo de la especialidad	Especialidad, sucursal de atencion y estado(activo o en guardia)
Sucursal/Especialidad	Catalogo	Sedes y especialidades que dispone el centro medico	Se muestran las sedes y especialidades activas

5. Requisitos funcionales de nivel 0 para el CRUD

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir al paciente agendar una cita médica en línea las 24 horas mediante el bot de mensajería, seleccionando sucursal, especialidad, médico, fecha y hora con cupos reales, y confirmando el resumen antes de registrarla.	Paciente	Agendar cita en línea
RF-02	El sistema debe permitir a la recepcionista registrar a los pacientes que acuden de forma presencial sin cita previa, validando sus datos y asignándoles un turno en su sucursal	Recepcionista	Registrar paciente presencial
RF-03	El sistema debe permitir al medico registrar, actualiza o bloquea su disponibilidad semanal según sus turnos o guardias	Medico	Gestionar disponibilidad medica
RF-04	El sistema debe permitir consultar las citas agendadas, al medico su agenda del dia y al administrador todas las citas con filtros como la sucursal, especialidad, medico, fecha y estado	Administrador/ medico	Consultar citas agendadas

6. Procedimiento para construir el diagrama en PlantUML

1. Revisar la ERS/SRS(Propuesta 2 v3.1) y seleccionar 4(cuatro) requisitos funcionales de nivel 0: RF-01, RF-02,RF-03, RF-04.
2. Identificar actores primarios, secundarios y sistemas externos. En AgendaMed Renacer los actores son Paciente, Recepcionista, Medico y Administrador
3. Redactar casos de uso con nombres claros orientados al objetivo del usuario: Agendar cita en linea, Registrar paciente presencial, Gestionar disponibilidad medica y Consultar citas agendadas.
4. Dibujar el límite del sistema para separar lo que pertenece a la solución de lo que está fuera.
5. Agregar relaciones actor-caso de uso y, solo si se justifica, relaciones <<include>> o <<extend>>.
6. Construir una matriz de trazabilidad RF-CU para verificar que ningún caso de uso aparezca sin requisito asociado.
7. Validar que el código PlantUML funcione y que el diagrama generado sea comprensible.

7. Código PlantUML: diagrama de casos de uso AgendaMed

```
@startuml CU_Nivel0_AgendaMed
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Paciente" as PAC
actor "Administrador" as ADM
actor "Recepcionista" as REC
actor "Médico" as MED

rectangle "AgendaMed Renacer - Bot de agendamiento (Nivel 0)" {
  usecase "Agendar cita\nen línea" as UC1
  usecase "Registrar paciente\npresencial" as UC2
  usecase "Gestionar disponibilidad\nmédica" as UC3
  usecase "Consultar citas\nagendadas" as UC4
  usecase "Validar disponibilidad\nde horario" as UC5
  usecase "Validar datos\ndel paciente" as UC6
  usecase "Derivar a\nrecepcionista" as UC7
}

PAC -- UC1
ADM -- UC1
ADM -- UC2
ADM -- UC3
ADM -- UC4
UC1 -- REC
UC2 -- REC
UC3 -- MED
UC4 -- MED

UC1 ..> UC5 : <<include>>
UC2 ..> UC6 : <<include>>
UC7 ..> UC1 : <<extend>>
@enduml
```

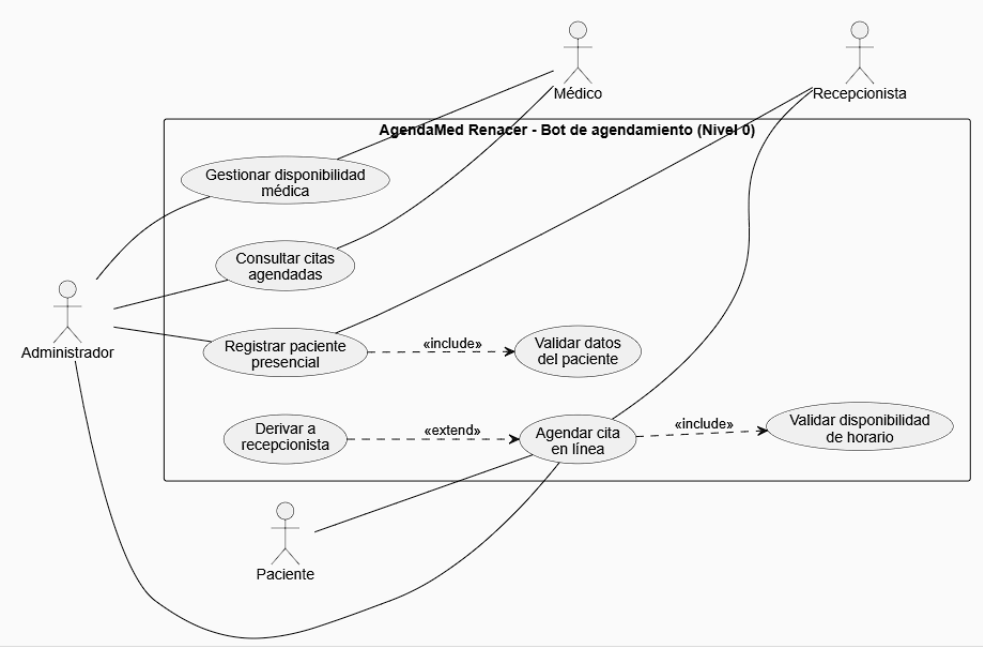


Figura 1. Diagrama de casos de uso de nivel 0

Interpretación: El Paciente ejecuta la funcion principal del Centro Medico(Agendar cita en linea); La Recepcionista atiende a los pacientes presenciales y recibe las conversaciones derivadas; El Medico gestiona la disponibilidad y consulta su agenda y el Administrador supervisa los cuatro procesos. Validar disponibilidad de horario se modela como <<include>> porque toda cita exige comprobar que la franja horaria este libre; Validar datos del paciente es <<include>> de Registrar paciente presencial porque la identificacion siempre se valida antes de guardar; Derivar a recepcionista es <<extend>> porque solo ocurre bajo condicion.

8. Especificación breve de casos de uso

Caso de uso	Actor	Precondición	Flujo principal resumido	Resultado esperado
Agenda cita en linea	Paciente	El paciente escribe al numero del centro medico y se registra; selecciona la sucursal y medicos con disponibilidad	Enviar mensaje inicial -> seleccionar sucursal -> seleccionar especialidad -> seleccionar medico -> seleccionar fecha y hora -> confirma resumen -> crea cita	Cita registrada con un ID unico en estado de pendiente_pago y bloqueada la franja de ese horario para el otros pacientes.
Registrar paciente presencial	Recepcionista	Recepcionista autenticada en el Dashboard de su sucursal para pacientes sin previa cita	Busca paciente -> ingresar o actualizar datos -> validar datos -> asignar turno -> guardar	Paciente registrado y turno presencial asignado el cual sera visible para el doctor
Gestionar disponibilidad medica	Medico	Medico autenticado en su panel, verifica el horario y la franja de citas reservadas	Abrir agenda -> registrar, editar o bloquear franjas -> validar horario de citas -> guardar	Disponibilidad actualizada, el bot mostrara solo cupos con una franja disponible
Consultar citas agendadas	Administrador/medico	Usuario autenticado, existen citas registradas	Seleccionar fecha y filtros -> consultar citas -> mostrar listado	Agenda visible en tiempo real, el medico ve solo sus citas y el administrador ve todas las citas

9. Puente hacia clases de análisis: frontera, control y entidad

Aunque esta guía se enfoca en casos de uso, los documentos base indican que el análisis orientado a objetos puede continuar identificando clases de frontera, control y entidad. Esto ayuda a preparar el paso hacia U1T2 y U1T3.

Tipo de clase	Propósito en análisis	Capa MVC	Ejemplo para AgendaMed	Sintaxis PlantUML sugerida
Frontera / Boundary	Representa la interacción con el actor o una pantalla del sistema.	Vista	InterfazChatbot, FormularioPacientePresencial, PanelDisponibilidad, PanelAgendaCitas	Boundary “InterfazChatBot” as UI
Control / Control	Coordina la lógica del caso de uso y la comunicación entre frontera y entidad.	Controlador	ControlAgendamiento, ControlPaciente, ControlDisponibilidad, ControlConsultaCitas	control "ControlAgendamiento" as Ctrl
Entidad / Entity	Representa información del dominio que el sistema conserva.	Modelo	Cita, Paciente, Disponibilidad	entity "Cita" as Cit
Repositorio	Conserva y recupera registro	Modelo(Persistencia)	RepositorioCita, RepositorioPaciente, RepositorioDisponibilidad	Database “RepositorioCita” as Repo

9.1. Puente 1: Agendar cita en linea(RF-01)

Actor: Paciente

Boundary(Vista): InterfazChatBot

Control(Controlador): ControlAgendamiento

Entity(Modelo): Cita

Repositorio: RepositorioCita

```
@startuml Puente_CU1_AgendarCitaEnLinea
left to right direction
actor "Paciente" as PAC
boundary "InterfazChatBot" as UI
control "ControlAgendamiento" as Ctrl
entity "Cita" as Cit
database "RepositorioCita" as Repo

PAC --> UI : solicita agendar cita (sucursal, especialidad, médico, fecha, hora)
UI --> Ctrl : envía sucursalId, especialidadId, medicoId, fecha y hora
Ctrl --> Cit : valida disponibilidad y crea la cita (PENDIENTE_PAGO)
Ctrl --> Repo : guarda la cita y bloquea la franja horaria
Ctrl --> UI : muestra resumen, código RENACER-XXXXX e instrucciones de pago
@enduml
```

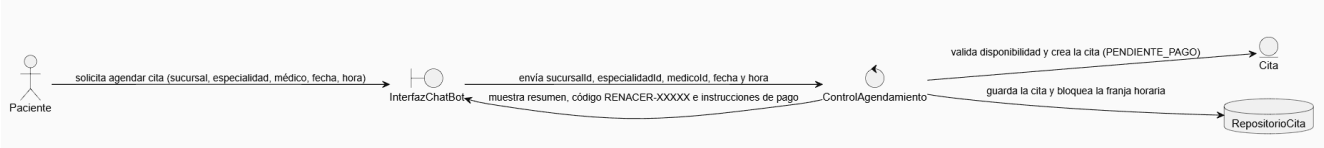


Figura 2. Puente 1: Agendar cita en linea

9.2. Puente 2: Registrar paciente presencial(RF-02)

Actor: Recepcionista

Boundury(Vista): FormularioPacientePresencial

Control(Controlador): ControlPaciente

Entity(Modelo): Paciente

Repositorio: RepositorioPaciente

```
@startuml Puente_CU2_RegistrarPacientePresencial
left to right direction
actor "Recepcionista" as REC
boundary "FormularioPacientePresencial" as UI
control "ControlPaciente" as Ctrl
entity "Paciente" as Pac
database "RepositorioPaciente" as Repo

REC --> UI : ingresa identificación y datos del paciente walk-in
UI --> Ctrl : envía identificación, nombres, teléfono, correo y consentimiento
Ctrl --> Pac : valida cédula/pasaporte/RUC y crea o actualiza el paciente
Ctrl --> Repo : guarda el paciente y su turno en la sucursal
Ctrl --> UI : confirma registro y turno asignado
@enduml
```

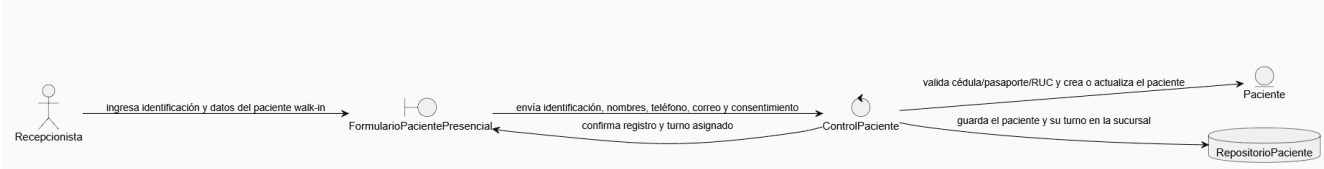


Figura 3. Puente 2: Registrar paciente presencial

9.3. Puente 3: Gestionar disponibilidad medica(RF-03)

Actor: Medico

Boundury(Vista): PanelDisponibilidad

Control(Controlador): ControlDisponibilidad

Entity(Modelo): Disponibilidad

Repositorio: RepositorioDisponibilidad

```
@startuml Puente_CU3_GestionarDisponibilidadMedica
left to right direction
actor "Médico" as MED
boundary "PanelDisponibilidad" as UI
```

```
control "ControlDisponibilidad" as Ctrl
entity "Disponibilidad" as Disp
database "RepositorioDisponibilidad" as Repo

MED --> UI : registra, edita o bloquea franjas de su semana
UI --> Ctrl : envía medicoId, sucursalId, día, hora inicio, hora fin y estado
Ctrl --> Disp : valida contra horario global y citas ya reservadas
Ctrl --> Repo : guarda o actualiza la disponibilidad
Ctrl --> UI : muestra la agenda semanal actualizada
@enduml
```

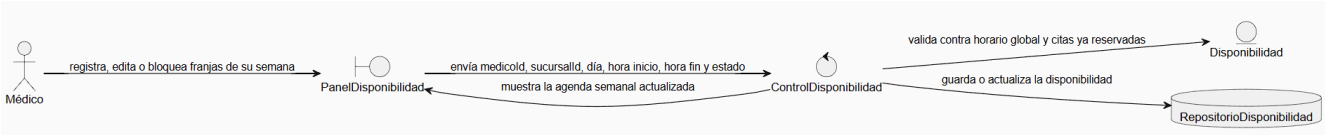


Figura 4. Puente 3: Gestionar disponibilidad medica

9.4. Puente 4: Consultar citas agendadas(RF-04)

Actor: Medico

Boundury(Vista): PanelAgendasCitas

Control(Controlador): ControlConsultaCitas

Entity(Modelo): Cita

Repositorio: RepositorioCita

```
@startuml Puente_CU4_ConsultarCitas
left to right direction
actor "Médico" as MED
boundary "PanelAgendaCitas" as UI
control "ControlConsultaCitas" as Ctrl
entity "Cita" as Cit
database "RepositorioCita" as Repo

MED --> UI : solicita ver su agenda (fecha y filtros)
UI --> Ctrl : envía medicoId, fecha, sucursal y estado
Ctrl --> Repo : consulta las citas que cumplen los filtros
Ctrl --> Cit : obtiene paciente, hora, sucursal y estado de cada cita
Ctrl --> UI : muestra el listado de citas agendadas
@enduml
```

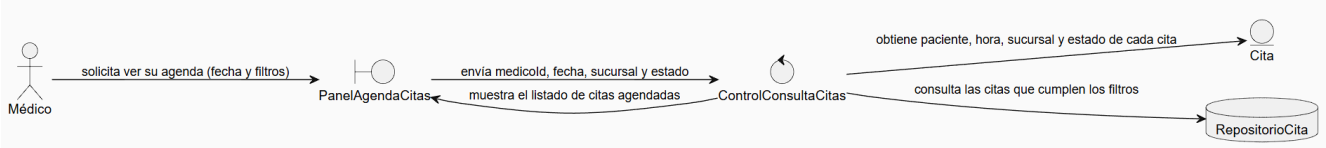


Figura 5. Puente 4: Consultar citas agendadas

10. Matriz de trazabilidad RF-CU para AgendaMed

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Agendar cita en linea	Paciente	Critica	El sistema registra una cita con paciente, medico, especialidad, sucursal, fecha y hora validas con ID unico y estado(segun el caso), sin colision de franjas y envia el resumen	UC1
RF-02	Registrar paciente presencial	Recepcionista	Alta	El sistema registra un paciente sin cita previa con identificacion valida, sin duplicar informacion y asigna un turno	UC2
RF-03	Gestionar disponibilidad medica	Medico	Critica	El sistema guarda las franjas del medico solo si estan dentro del horario de atencion y no se chocan las citas reservadas	UC3
RF-04	Consultar citas agendadas	Administrador/ medico	Alta	El sistema muestra las citas que cumplen con los filtros, paciente, hora, sucursal y estado	UC4
RF-01	Validar disponibilidad de horario (<<include>>)	Sistema	Critica	El sistema impide reservar una franja ocupada o fuera del horizonte de 30 dias	UC5
RF-02	Validar datos de paciente (<<include>>)	Sistema	Critica	El sistema impide guardar pacientes con identificacion invalida, correo invalido o sin previo consentimiento	UC6
RF-03	Derivar a recepcionista (<<extend>>)	Recepcionista	Alta	Ante la solicitud de un paciente y 3 errores o timeout, el chat pasa a HUMAN_HANDOFF o PENDIENTE_REVISION	UC7

Las filas UC5, UC6 y UC7 corresponden a los casos de uso de apoyo (<<include>>/<<extend>>) cada uno conserva un requisito de origen en el SRS de modo que ningún caso de uso del diagrama queda sin requisito asociado

Conclusión: La trazabilidad, la consistencia y la utilidad del modelo permiten que el proyecto AgendaMed Renacer traduzca de forma sencilla las necesidades de la clínica en un diseño claro y bien estructurado. Por un lado, la trazabilidad asegura que cada requisito del proyecto se corresponda exactamente con un caso de uso en PlantUML, evitando que alguna regla de negocio quede en el aire. Al mismo tiempo, la consistencia logra que todo el diseño mantenga un orden lógico y coherente, separando claramente lo que ve el usuario, la lógica que procesa la información y la base de datos donde se guarda todo.

Por su parte, la utilidad del modelo está en su capacidad para convertir explicaciones teóricas en un mapa visual que el equipo puede entender, ajustar y mantener fácilmente mediante código. Al unir la trazabilidad de los requisitos con la consistencia de la estructura, el modelo sirve como un puente directo de comunicación entre los analistas y los programadores, alineando a todos sobre lo que se debe construir y facilitando las siguientes etapas del proyecto.

11. Lista de verificación para revisar el diagrama

- Cada caso de uso se deriva de un requisito funcional de nivel 0.
- El actor representa un rol externo, no una pantalla ni una clase interna.
- El nombre del caso de uso está redactado como objetivo del usuario.
- El límite del sistema está claramente identificado.
- Las relaciones <<include>> o <<extend>> tienen justificación funcional.
- El diagrama no modela detalles técnicos prematuros.
- El código PlantUML compila y genera un diagrama legible.
- La matriz RF-CU permite verificar trazabilidad sin pérdida de información.

12. Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad RF-CU	Relación clara entre requisitos funcionales, actor, caso de uso y criterio de aceptación.	5 pts	Matriz RF-CU completa.
Modelado de casos de uso	Representación correcta de actor, límite del sistema, casos de uso y relaciones.	5 pts	Diagrama PlantUML generado.
Claridad del análisis	Selección pertinente de funcionalidades de nivel 0 y nombres orientados a objetivos del usuario.	4 pts	Casos de uso redactados claramente.
Uso correcto de PlantUML	Código funcional, ordenado y comprensible.	3 pts	Código sin errores de sintaxis.
Presentación técnica	Documento ordenado, con evidencias, explicación breve y coherencia visual.	3 pts	Entrega final documentada.

13. Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Requisitos funcionales	Listado de RF de nivel 0 para el CRUD de Estudiante.	Tabla RF.
Diagrama de casos de uso	Código PlantUML y captura del diagrama generado.	Código + imagen.
Matriz de trazabilidad	Relación RF, actor, caso de uso, prioridad y criterio de aceptación.	Tabla RF-CU.
Conclusión técnica	Reflexión breve sobre trazabilidad, consistencia y utilidad del modelo.	Texto de cierre.

14. Fuentes internas consultadas

Documentos revisados desde la carpeta de Google Drive compartida por la docente:

- Modelado de Clases de Análisis en PlantUML.docx
- Tareas_U1_Autonomo_Analisis_Diseno_OO_PlantUML.docx

Nota: No se consultó información de internet para elaborar esta guía, de acuerdo con la instrucción recibida.